Prepared by: Kirolos Mina
Supervised by: Dr. Rami Zewail

CSE 521: Embedded ML

Edge AI lab & TinyML

Content	Page
Overview	1
Edge AI software tools for different devices	2
Getting started with Edge AI MPUs	3
Getting started with tinyml	5

➢ Overview

- Edge AI refers to running AI and ML models directly on local hardware devices (like smartphones, smart cameras, or appliances) rather than in a remote cloud data center.
- TinyML is a specialized subset of Edge AI. It focuses on shrinking these AI models so they can run on ultra-small, low power microelectronics.
- This offers three major benefits:
  - zero latency
  - enhanced privacy
  - offline functionality

➢ Edge AI software tools for different devices

Texas instrument is a pioneering company in EdgeAI field, and has its own software tools within its ecosystem, the tools are as follows:

1- CCStudio Edge AI Studio:

- Uses: it unifies data collection, model training, performance analysis and deployment. It handles everything from simple visual steps to complex command-line processes.
- Key components: model composer and model analyzer
- Model composer: script-based tool used to collect and annotate data, train neural networks and compile models directly for target devices.
- Model analyzer: cloud-based service that allows developers to benchmark deep learning models on remote development boards in minutes, displaying metrics like FPS and memory usage

2- TI deep learning (TIDL) tools

- Uses: It allows developers to run machine learning models directly on specialized on-chip accelerators (such as digital signal processor or a deep learning matrix accelerator)

- Run-time-support: provides off-the-shelf support for industry-standard frameworks like ONNX and TensorFlow lite.

3- Edge AI Model Zoo

- Vast, built-in repository containing pre-trained models optimized for TI's hardware
- Uses: Instead of building a model from scratch, developers can choose from dozens of pre-trained architectures for tasks like image classification, semantic segmentation and object detection.

4- Processor SDK linux for Edge AI

- A scalable software development kit (SDK) providing pre-tuned inference engines and foundational AI applications.
- Uses: it acts as the software foundation to connect camera streams or sensor data to live display outputs. It features plug-and-play integration with tools like Gstreamer, OpenCV and OpenVX

These tools are compatible with MCUs and MPUs

➢ Getting started with Edge AI MPUs

- Overview: to use AM6xA and TDA4X processors for your Edge AI system, follow these steps:

  1- Select a processor and acquire evaluation hardware

2- Setup the SDK for your device, install to an SD card and boot the Edge AI SDK on the evaluation hardware

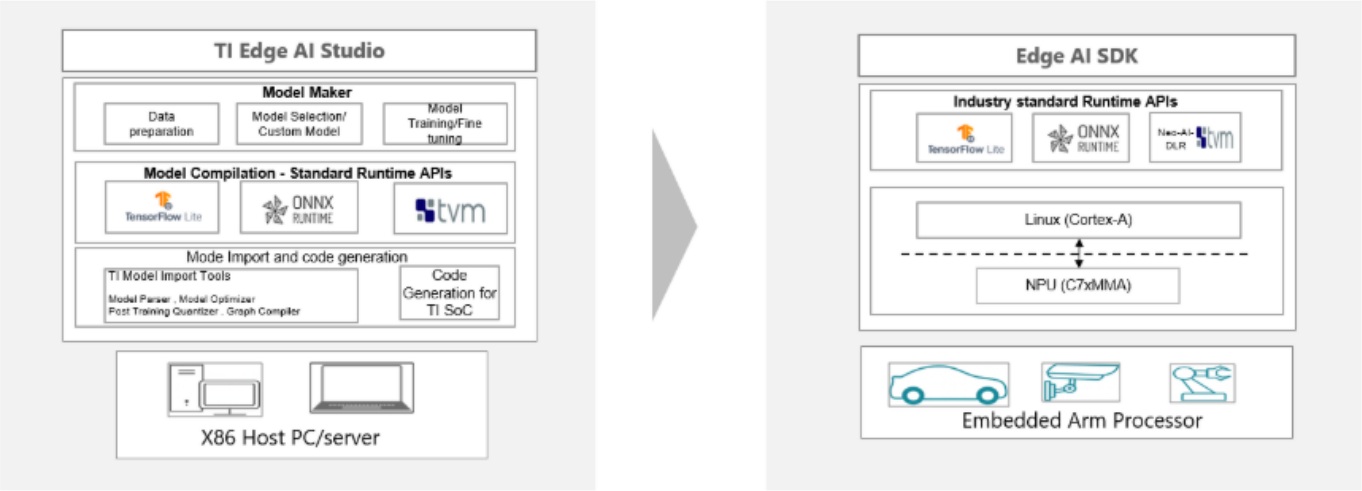
3- Use TIDL to compile neural network models for the C7 AI accelerator and run on the target device.

4- Run models with open-source runtimes and integrate into an end-to-end application with Gstreamer or TI's OpenVX implementation.

TI Edge AI model development flow

At a high-level, the Edge AI development flows features two stages.

1. On x86 host machine (PC or server), import and compile a trained model for the C7 NPU
   - Compiled models can be run on PC with a bit-accurate emulator of the C7 NPU to evaluate and optimize accuracy
2. Transfer your compiled model to the embedded processor and accelerate with standard runtimes.



TI provides tools for various points in the process.

- For each GUI-based tool, there is a corresponding open source, programmatic / command-line tool

  After initial evaluation of a model architecture, developers will often iterate on their model to achieve optimal performance and accuracy.

- The first stage of performance optimization is by ensuring all layers within a model have acceleration support. Accuracy optimization can be approached during or after model training by post-training quantization and quantization aware-training

➢ Getting started with TinyML

Installation: the project uses modern python tooling with hatch for dependency management and development workflows

```
# Clone the repository
git clone https://github.com/umitkacar/ai-edge-computing-tiny-embedded.git
cd ai-edge-computing-tiny-embedded

# Install dependencies (using hatch)
pip install hatch

# Run tests
hatch run test

# Run full CI pipeline
hatch run ci
```

Development setup

```
# Linting & Formatting
hatch run lint           # Run Ruff linter
hatch run format         # Format code with Black
hatch run format-check   # Check formatting without changes

# Type Checking
hatch run type-check     # Run Mypy strict type checking

# Testing
hatch run test                    # Run tests (sequential)
hatch run test-parallel           # Run tests with auto workers
hatch run test-parallel-cov       # Parallel tests with coverage

# Security
hatch run security       # Run Bandit security audit

# Complete CI Pipeline
hatch run ci           # Run all checks (format, lint, type-check, security
```

Features

- ✓ INT8 Quantization
- ✓ INT4 Quantization
- ✓ FP16 Quantization
- ✓ Dynamic Quantization
- ✓ Symmetric and asymmetric modes
- ✓ Per-tensor & per-channel quantization
- ✓ Weight quantization with 6 different modes
- ✓ Compression ratio analysis
- ✓ Model size calculation
- ✓ Type-safe APIs with full annotations

✓Comprehensive error handling

Example usage

```python
import numpy as np
from ai_edge_tinyml import Quantizer, QuantizationConfig, QuantizationMode

# Create quantization config
config = QuantizationConfig(
    mode=QuantizationMode.INT8,
    symmetric=True,
    per_channel=False
)

# Initialize quantizer
quantizer = Quantizer(config)

# Quantize weights
weights = np.random.randn(100, 100).astype(np.float32)
quantized = quantizer.quantize(weights)

# Dequantize for inference
dequantized = quantizer.dequantize(quantized)

# Calculate compression
from ai_edge_tinyml.utils import calculate_compression_ratio
ratio = calculate_compression_ratio(weights, quantized)
print(f"Compression ratio: {ratio:.2f}x")
```